Saarthi Manual

Section 6: Voice Optimization + Smart Alert Management

🎯 6.1 OBJECTIVES

By the end of this section, you will have:
✅ Clean, loud, and clear voice alerts
✅ Fewer repeated or annoying warnings
✅ Customized speed, pitch, and volume
✅ Optional Hindi or Indian English voice
✅ Smarter logic to **avoid speaking the same object again and again**

🔊 6.2 WHY WE'RE IMPROVING THE VOICE SYSTEM

Without optimization:

- Saarthi might **repeat** the same word ("Dog ahead", "Dog ahead"...).
- The default `espeak` voice is robotic and fast.
- Users might not understand alerts in loud or busy environments.

We're going to solve all of this step by step.

🔧 6.3 TEXT-TO-SPEECH (TTS) OPTIONS FOR RASPBERRY PI

TTS Engine	Offline?	Language Support	Speed Control	Indian Voice
espeak	✅ Yes	English only	✅	❌
pyttsx3	✅ Yes	English, Hindi	✅	✅* (depends on OS)
gTTS	❌ No	50+ languages	✅	✅

➡️ We'll use **`pyttsx3`** because:

- It's offline
- Better voice quality
- Customizable voice, pitch, speed

🪢 6.4 INSTALL `pyttsx3`

```
pip3 install pyttsx3
sudo apt install espeak-ng
```

□ 6.5 CREATE A SMART AUDIO MODULE

📄 `audio.py` – Updated with `pyttsx3`

```
# audio.py
import pyttsx3
import time

engine = pyttsx3.init()
engine.setProperty('rate', 140)  # Speed
engine.setProperty('volume', 1.0)  # Max volume

# Optional: set Indian English voice
voices = engine.getProperty('voices')
for voice in voices:
    if 'english' in voice.name.lower() and 'india' in voice.name.lower():
        engine.setProperty('voice', voice.id)
        break

last_spoken = {}
timeout = 4  # seconds between repeating same message

def speak(text):
    now = time.time()
    if text not in last_spoken or now - last_spoken[text] > timeout:
        print(f"[VOICE] {text}")
        engine.say(text)
        engine.runAndWait()
        last_spoken[text] = now
```

□ 6.6 INTELLIGENT ALERT FILTERING

Problem:

Saarthi might say "person ahead" again and again, even if the person is standing still.

Solution:

We store the **last spoken message and a timestamp**, and prevent repeating the same thing within X seconds.

Already implemented above using:

```
last_spoken = {}
timeout = 4
```

If the same message comes within 4 seconds, it is skipped.

🔉 6.7 CUSTOMIZING VOICE (OPTIONAL)

To list available voices:

```
import pyttsx3
engine = pyttsx3.init()
voices = engine.getProperty('voices')
for i, voice in enumerate(voices):
    print(f"{i}: {voice.name}")
```

Then set:

```
engine.setProperty('voice', voices[INDEX].id)
```

📣 6.8 ALERT PHRASE ENHANCEMENTS

Make Saarthi more natural and understandable:

Situation	Default	Better Spoken Phrase
Object ahead	"Dog ahead"	"Careful, a dog is ahead"
Close object	"Obstacle very close"	"Warning! Obstacle ahead!"
Stair detected	"Stair down detected"	"Watch out! Stairs going down"

You can modify the phrases directly in `main.py`:

```
speak(f"Watch out! {obj} ahead")
```

□ 6.9 TEST OUTPUT + IMPROVEMENTS

1. Walk in front of the camera with your hand or a ball. It should say:

“Watch out! Person ahead” (once every ~4 seconds)

2. Simulate stairs by removing object under downward sensor. It should say:

“Watch out! Stairs going down”

3. Place a bag or chair 30cm away. It should say:

“Warning! Obstacle ahead!”

✅ 6.10 SECTION SUMMARY

- You’ve now integrated **pyttsx3** for clearer offline voice
- Customized for:
  - **Indian English voice**
  - **Pacing, speed, volume**
- Implemented **smart repetition filtering**
- Created a **smooth, calm but responsive alert system**

➔ SOON NEXT SECTION:

Section 7 – Waterproofing, Enclosure Design, and Hardware Mounting

In the next section, we'll handle:

- Where to physically place each part (camera, Pi, sensors)
- How to waterproof and protect the setup
- Building a strong, field-ready prototype